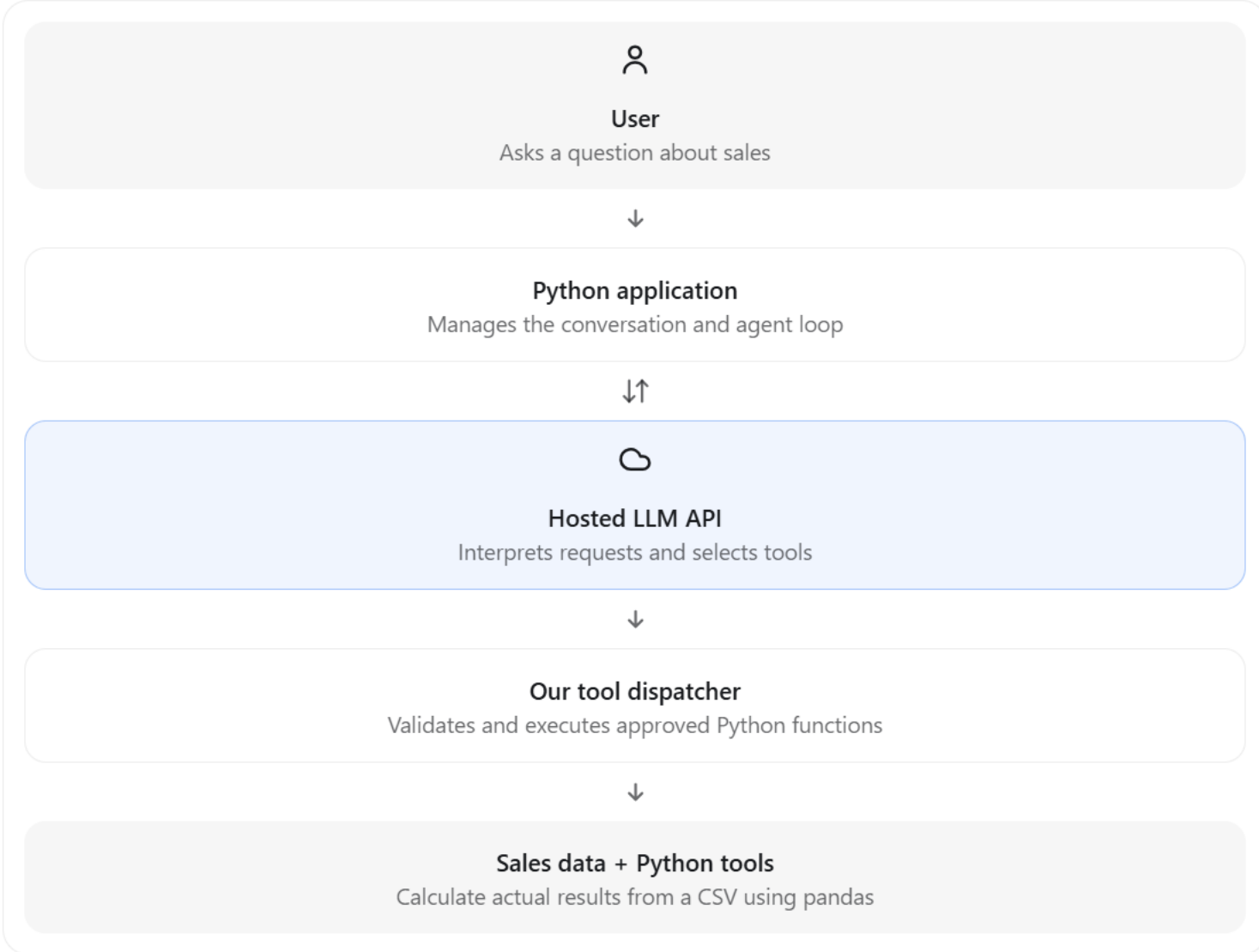
AI Agent 🤖 from Scratch

Naji El Kotob

v.1.1.6 – 12 October 2026

Build an AI Agent from Scratch

- Build a **functional AI agent step by step using Python**, a hosted LLM API, and real business data.
- Starting with a simple LLM interaction, progressively implement tool calling, tool execution, and the agent control loop from scratch - without agent frameworks.
- Using a sales CSV as the business scenario, learn how an LLM interprets user requests, selects tools, receives execution results, and produces evidence-based answers.
- The project emphasizes deep conceptual understanding, transparent implementation, and line-by-line explanations of **how AI agents work**.
- Technologies: Python, OpenAI Python SDK (with Groq), pandas, CSV.
- Learning outcome: Understand and build a tool-calling AI agent from the ground up, including its core architecture, execution loop, and essential safeguards.


User
Asks a question about sales
Python application
Manages the conversation and agent loop
Hosted LLM API
Interprets requests and selects tools
Our tool dispatcher
Validates and executes approved Python functions
Sales data + Python tools
Calculate actual results from a CSV using pandas

Technology stack

Component	Choice
Language	Python
LLM	Hosted API with reliable tool-calling support
Initial provider	Groq, through the OpenAI Python SDK (model openai/gpt-oss-120b)
Data	Sample sales CSV
Calculations	pandas
Interface	Terminal
Agent framework	None
Development environment	VS Code or another Python editor

What you should understand deeply

Concept	In plain words	Stage	File
LLM interaction	Send messages with roles (system, user, assistant, tool) and read the reply	1, 7	main.py, llm.py
Tools	Describe a function by name, description and parameters so the LLM knows when to use it	2, 3, 5	tools.py, tool_schemas.py
Tool execution	The LLM only requests a tool – our Python app runs it	4	run_tool_call.py
The agent loop	Return results to the LLM; it decides: another tool or the final answer	6	agent.py
State	Keep the conversation and tool results in the messages list across calls	7	agent_chat.py
Reliability	Validate arguments, handle errors, limit steps, ask when unclear	8	agent_reliable.py

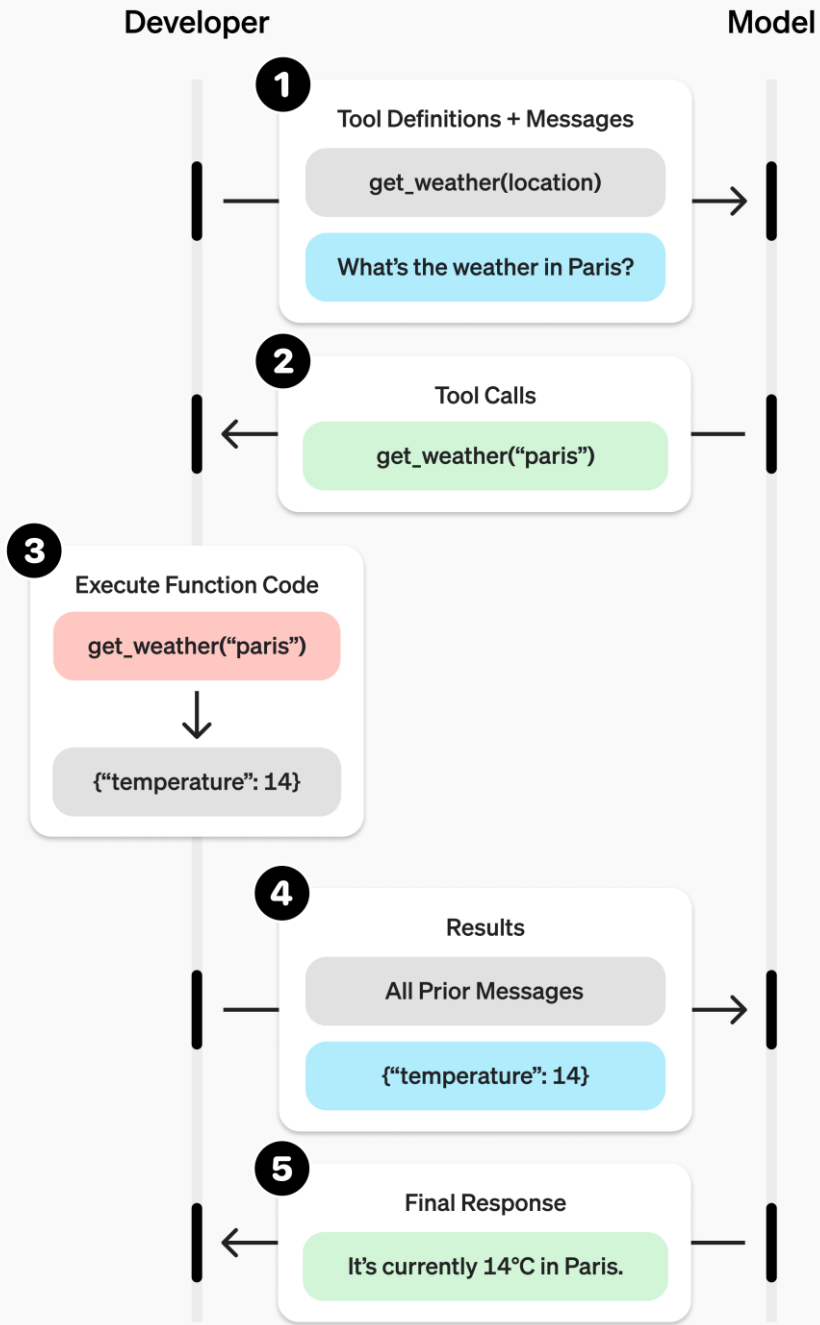
The implementation strategy

- Stage 1: Ask the LLM a question and display its response.
- Stage 2: Create a Python sales-analysis function without any LLM.
- Stage 3: Let the LLM request that function through tool calling.
- Stage 4: Execute the requested tool ourselves and return the result.
- Stage 5: Add more tools with parameters and let the LLM choose.
- Stage 6: Build the agent loop: repeat until the answer is complete.
- Stage 7: Add instructions and memory across questions.
- Stage 8: Make it reliable: validation, errors, limits and clarification.

Function calling

- Give models access to new functionality and data they can use to follow instructions and respond to prompts.

- [Function calling | OpenAI API]



Create the project

- **mkdir** ai-agent-from-scratch
- **cd** ai-agent-from-scratch
- **python** -m venv .venv

Activate the virtual environment

🪟 .venv\Scripts\Activate.ps1

🍏 source .venv/bin/activate

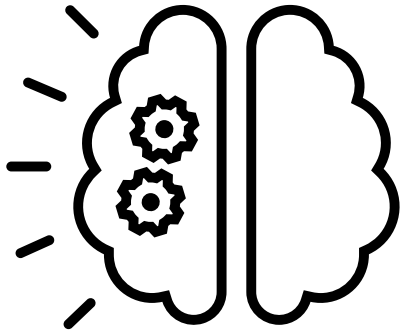
A virtual environment keeps this project's Python packages separate from those of other projects. Run **deactivate** to leave it. If you ever want to start fresh, you can simply delete the .venv folder and recreate it.

Install the OpenAI Python SDK

- python -m pip install **openai**

Protect your API key

- $env:AI_API_KEY="your-api-key-here"
- export AI_API_KEY="your-api-key-here"



1. Write our first LLM program

Talk to a hosted LLM from Python.

Let’s talk about LLMs

main.py

```
import os
from openai import OpenAI

api_key = os.getenv("AI_API_KEY")

client = OpenAI(api_key=api_key,
base_url="https://api.groq.com/openai/v1")

response = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[{"role": "user",
               "content": "Explain what revenue means in business in two
sentences."}],
)

print(response.choices[0].message.content)
```

llm.py

```
import os
from openai import OpenAI

# Shared LLM setup – every demo file imports client and MODEL from here.
MODEL = "openai/gpt-oss-120b"

api_key = os.getenv("AI_API_KEY")

if not api_key:
    raise RuntimeError("AI_API_KEY is not configured.")

client = OpenAI(api_key=api_key, base_url="https://api.groq.com/openai/v1")
```



2. Give our agent something to do

A Python tool that calculates real numbers – no AI involved.

Our scenario

- A business manager asks:
  - What were our total sales, total profit, and profit margin?
- We will build a Python function that answers this question using a CSV file.
- For now, we won't involve the LLM.

Our sales dataset

OrderDate,Product,Category,Quantity,UnitPrice,UnitCost,Region
2026-01-05,Laptop,Electronics,2,1200,900,North
2026-01-08,Mouse,Accessories,10,25,10,South
2026-01-12,Monitor,Electronics,3,300,200,North
2026-02-03,Laptop,Electronics,1,1200,900,South
2026-02-10,Keyboard,Accessories,5,60,30,North
2026-02-15,Monitor,Electronics,2,300,200,West
2026-03-02,Mouse,Accessories,20,25,10,West
2026-03-10,Keyboard,Accessories,8,60,30,South
2026-03-18,Laptop,Electronics,2,1200,900,North
2026-03-22,Monitor,Electronics,4,300,200,South

Our sales dataset

- Each row represents an order line. Our business calculations are:
  - Revenue = Quantity × UnitPrice
  - Cost = Quantity × UnitCost
  - Profit = Revenue − Cost
  - Profit margin = Profit ÷ Revenue × 100

Install pandas 

- python -m pip install **pandas**

tools.py

```
import pandas as pd

def get_sales_summary():
    # Load the sales dataset.
    df = pd.read_csv("sales.csv")
    # Calculate revenue and cost for each order line.
    df["Revenue"] = df["Quantity"] * df["UnitPrice"]
    df["Cost"] = df["Quantity"] * df["UnitCost"]
    # Calculate profit for each order line.
    df["Profit"] = df["Revenue"] - df["Cost"]
    # Aggregate the results.
    total_revenue = df["Revenue"].sum()
    total_cost = df["Cost"].sum()
    total_profit = df["Profit"].sum()
    # Avoid division by zero.
    profit_margin = total_profit / total_revenue * 100 if total_revenue != 0 else 0
    # Return structured results.
    return {
        "total_revenue": round(float(total_revenue), 2),
        "total_cost": round(float(total_cost), 2),
        "total_profit": round(float(total_profit), 2),
        "profit_margin_percent": round(float(profit_margin), 2),
    }
```

Understand the key design decision

- Notice that this function returns a dictionary, rather than printing a formatted answer.

Why?

- Because the results may eventually be consumed by several different components:
  1. The terminal can display the numbers.
  2. The LLM can interpret the results and explain them.
  3. A web interface can render them as cards or charts.
  4. Another function can use them for further calculations.
- Separating computation from presentation makes the tool reusable.

Test the tool independently

Create test_tool.py:

```
from tools import get_sales_summary

result = get_sales_summary()

print(result)
```

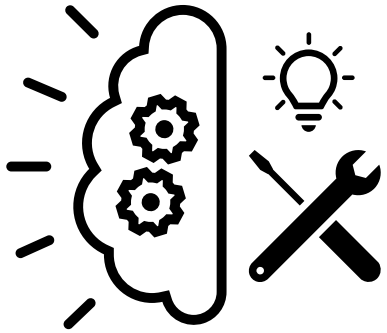
Output:

```
{'total_revenue': 10230.0, 'total_cost': 6990.0, 'total_profit': 3240.0, 'profit_margin_percent': 31.67}
```

What have we learned?

We now have three distinct elements:

Element	Responsibility
LLM	Understands natural language and makes decisions
Python tool	Performs deterministic business calculations
CSV	Supplies the underlying business data



3. Teach the LLM about our tool

Describe the tool so the LLM can request it.

Teach the LLM about our tool

- Python already knew which function to execute. There was no AI decision involved.
- Now imagine asking:
  - What were our total sales and profit?
- We want the LLM to decide that get_sales_summary is the appropriate tool.
- To make that possible, we must give the LLM a description of the tool and its parameters.

inspect_tool_call.py

```
from llm import client, MODEL

# 1. Describe the tool (the LLM never sees our Python code, only this).
tools = [{
    "type": "function",
    "function": {
        "name": "get_sales_summary",
        "description": "Get total revenue, cost, profit and profit margin from the sales data.",
        "parameters": {"type": "object", "properties": {}, "additionalProperties": False},
    },
}]

messages = [{"role": "user", "content": "What were our total sales and profit?"}]

# 2. Ask the LLM – it decides whether to request the tool.
response = client.chat.completions.create(model=MODEL, messages=messages, tools=tools)
msg = response.choices[0].message

# 3. Inspect the decision. Nothing is executed here.
if msg.tool_calls:
    call = msg.tool_calls[0]
    print("LLM requested:", call.function.name)
    print("Arguments:    ", call.function.arguments)
    print("Request id:   ", call.id)
    print("\nNothing has been calculated yet. The LLM only asked.")
else:
    print("No tool was needed:", msg.content)
```

Run it

```
> python inspect_tool_call.py
```

```
(.venv) PS D:\OneDrive\Desktop\AI\Labs\ai-agent-from-scratch> python inspect_tool_call.py
LLM requested: get_sales_summary
Arguments:     {}
Request id:    fc_d3dd0043-a41e-4941-89f6-6562ff45e2a3

Nothing has been calculated yet. The LLM only asked.
```

Understand what we just told the LLM

- This is the critical separation:
  - **LLM: Decides which tool to request.**
  - Python application: Must execute the requested function.
  - Tool: Performs the actual calculation.

From request to result

Stage 3 printed three things. Stage 4 uses each one:

Stage 3 printed	Stage 4 uses it to…	Line in run_tool_call.py
`LLM requested: get_sales_summary`	pick the function to run	`available_tools[call.function.name]()`
`Arguments: {}`	pass the inputs (empty for now, used from stage 5)	`func(**args)`
`Request id: fc_d3dd...`	label our result as the answer to that request	`"tool_call_id": call.id`

The LLM asked. Stage 4 answers.

The roadmap: from LLM call to agent

1 First LLM call
Send a question, print the answer
`main.py`

→

2 Business tool
Real numbers from the CSV, no AI
`tools.py`

→

3 Describe the tool
The LLM requests the tool
`inspect_tool_call.py`

→

4 Execute the tool
Python runs it, returns the result
`run_tool_call.py`

5 Many tools
The LLM chooses and fills parameters
`multi_tools.py`

→

6 Agent loop
Repeat until the answer is complete
`agent.py`

→

7 State
Instructions + memory across questions
`agent_chat.py`

→

8 Reliability
Validate, handle errors, stop, ask
`agent_reliable.py`

Done: stages 1–3 (gray). Next: stage 4 (blue). Each stage adds ONE idea and keeps everything before it.

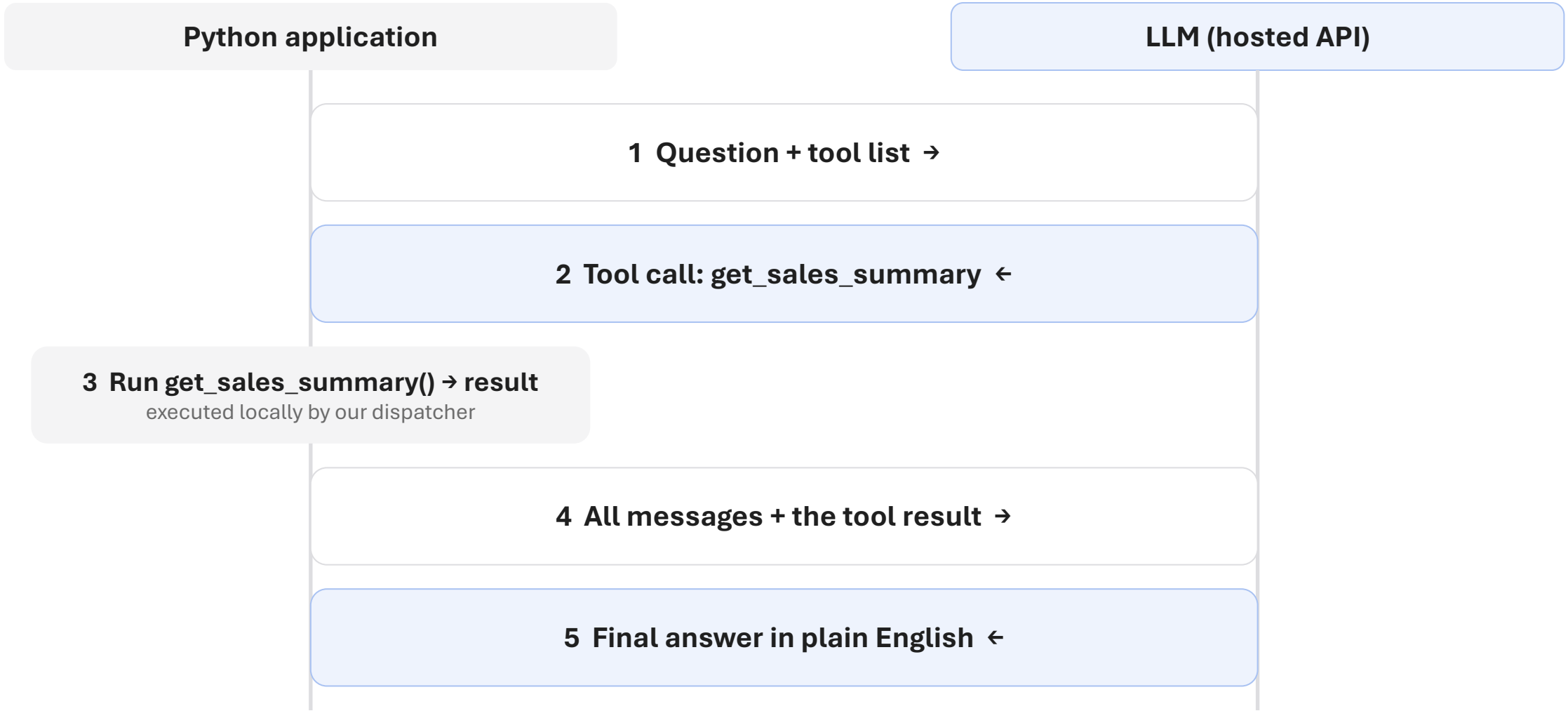
4. Execute the tool ourselves

Our app runs the requested tool and returns the result.

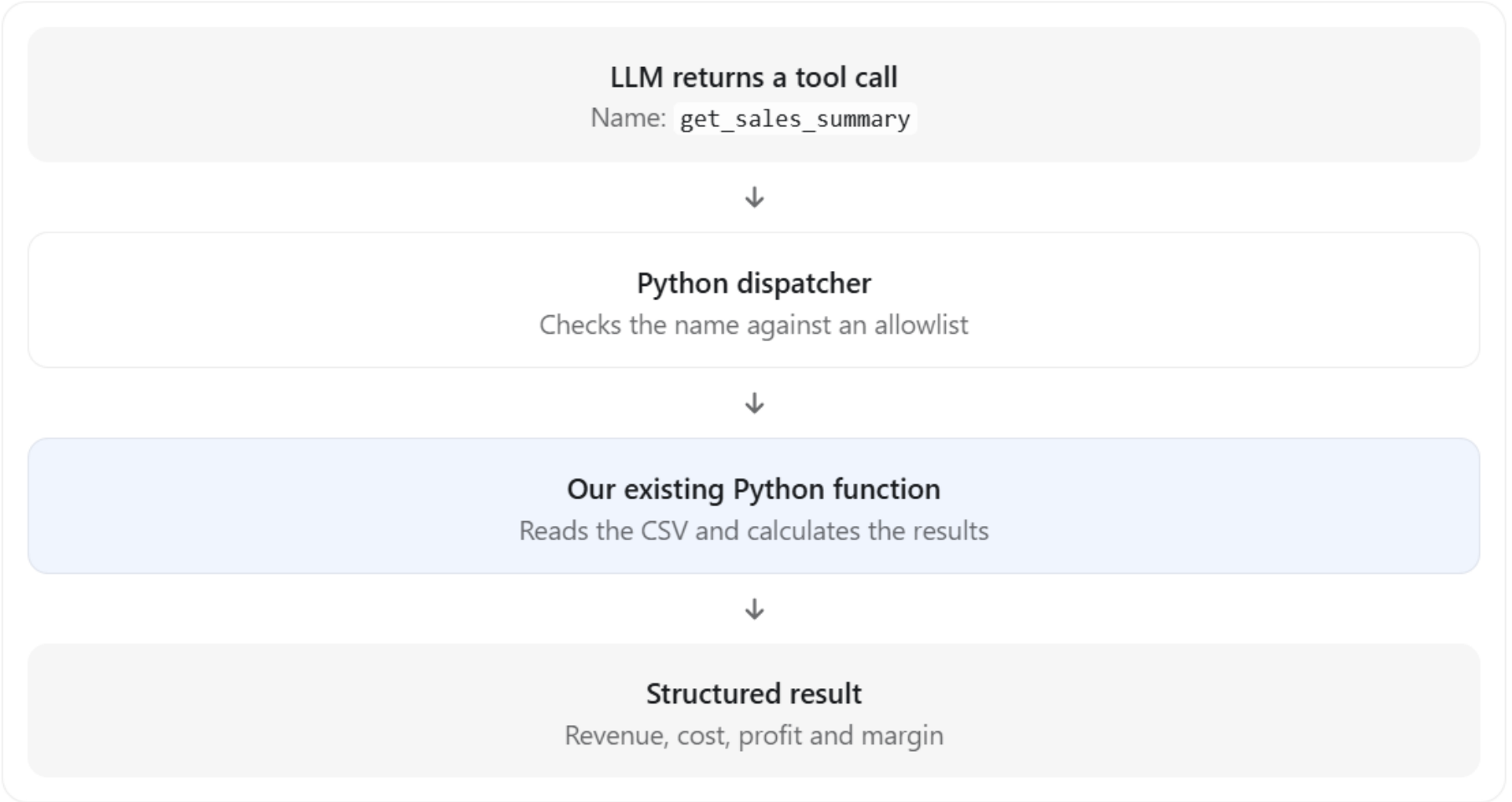
Why this stage?

- In stage 3 the LLM returned a **request**, not an answer
- The LLM cannot run code or open files – our Python app must:
  - 1. Find the requested function (the **dispatcher**)
  - 2. Run it and get the real numbers
  - 3. Send the result back to the LLM
  - 4. Ask the LLM to write the final answer
- The LLM never touches `sales.csv` – it only sees the result we give it

The round trip



Understand the flow



run_tool_call.py (1/2)

```
import json
from llm import client, MODEL
from tools import get_sales_summary

# The tool description (same as inspect_tool_call.py).
tools = [{ ... }]

# The dispatcher's allowlist: tool name -> real Python function.
available_tools = {"get_sales_summary": get_sales_summary}

messages = [{"role": "user", "content": "What were our total sales and profit?"}]

# Step 1 – the LLM decides.
response = client.chat.completions.create(model=MODEL, messages=messages, tools=tools)
msg = response.choices[0].message
```

run_tool_call.py (2/2)

```
if msg.tool_calls:
    call = msg.tool_calls[0]
    print("1. LLM requested:", call.function.name)

    # Step 2 – Python executes (the LLM cannot run code).
    result = available_tools[call.function.name]()
    print("2. Python ran it:", result)

    # Step 3 – return the result to the LLM.
    messages.append(msg)                                  # the LLM's request
    messages.append({"role": "tool",                      # our answer to it
                     "tool_call_id": call.id,
                     "content": json.dumps(result)})

    # Step 4 – the LLM writes the final answer from real numbers.
    final = client.chat.completions.create(model=MODEL, messages=messages)
    print("3. Final answer:\n", final.choices[0].message.content)
else:
    print("No tool was needed:", msg.content)
```

The tool message

How we hand the result back to the LLM:

Field	Value	Why
`role`	"tool"	Tells the LLM this message is a tool result, not a person
`tool_call_id`	call.id	Links the result to the exact request (matters with several calls)
`content`	json.dumps(result)	The result as text – messages only carry text

The conversation grows

user	`What were our total sales and profit?`
assistant	`tool_calls: get_sales_summary {}`
tool	`{"total_revenue": 10230.0, "total_profit": 3240.0, ...}`
assistant	`Total revenue was $10,230 and profit $3,240 (31.67% margin).`

Rows 1–3 are sent in the **2nd API call**.

Row 4 is what it returns.

The LLM keeps no memory between calls. Each call must carry the full conversation – our list does that.

Run it

```
> python run_tool_call.py
```

```
1. LLM requested: get_sales_summary
2. Python ran it: {'total_revenue': 10230.0, 'total_cost': 6990.0, 'total_profit': 3240.0,
'profit_margin_percent': 31.67}
3. Final answer:
 Total sales were $10,230 and total profit was $3,240 – a profit margin of 31.67%.

Conversation sent: ['user', 'assistant', 'tool']
```

Example output – the wording changes between runs. The numbers never do: they come from our tool.

What have we learned?

Stage 4 completes one full round trip:

Element	Responsibility
LLM	Decides which tool to request, then explains the result
Dispatcher (`available_tools`)	Maps the requested name to an approved function and runs it
Tool	Calculates the real numbers
`messages`	Carries the request and the result back to the LLM

5. Give the agent choices

Several tools with parameters – the LLM chooses.

Why this stage?

- One tool answers one kind of question
- Managers ask many: by region, by product, in another currency…
- We add two tools and let the LLM **choose** – and **fill the arguments**
- Our code barely changes: a longer menu and a `for` loop over the requests

tools.py (new tools)

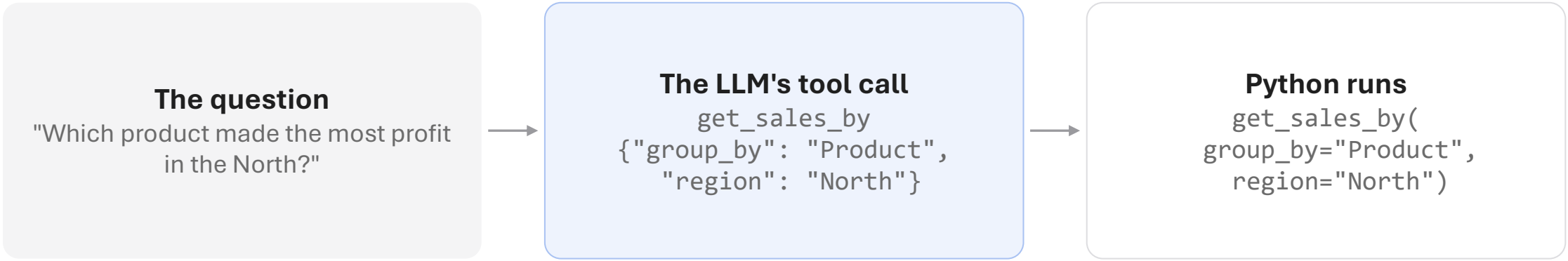
```
def get_sales_by(group_by, region=None):
    # Revenue and profit per Region, Product or Category, optionally for one region.
    if group_by not in ["Region", "Product", "Category"]:
        return {"error": f"Cannot group by '{group_by}'. Use Region, Product or Category."}
    df = pd.read_csv("sales.csv")
    if region:
        df = df[df["Region"].str.lower() == region.lower()].copy()
    df["Revenue"] = df["Quantity"] * df["UnitPrice"]
    df["Profit"] = df["Revenue"] - df["Quantity"] * df["UnitCost"]
    totals = df.groupby(group_by)[["Revenue", "Profit"]].sum().sort_values("Profit", ascending=False)
    return {"group_by": group_by, "region": region or "All regions", "rows": [ ... ]}


def get_exchange_rate(currency):
    # Dummy tool – fixed rates instead of a real currency API.
    rates = {"EUR": 0.92, "AED": 3.67, "GBP": 0.79}
    code = currency.upper()
    if code not in rates:
        return {"error": f"No rate available for '{currency}'."}
    return {"currency": code, "rate_vs_usd": rates[code]}
```

tool_schemas.py

```
# The tool "menu" we show the LLM. It never sees our Python code – only this.
TOOLS = [
    { ... get_sales_summary ... },
    {
        "type": "function",
        "function": {
            "name": "get_sales_by",
            "description": "Get revenue and profit per Region, Product or Category, "
                           "sorted from most to least profitable. Can be limited to one region.",
            "parameters": {
                "type": "object",
                "properties": {
                    "group_by": {"type": "string", "enum": ["Region", "Product", "Category"]},
                    "region": {"type": "string", "description": "Optional, e.g. North, South, West"},
                },
                "required": ["group_by"],
            },
        },
    },
    { ... get_exchange_rate ... },
]
```

Parameters: the LLM fills them



The LLM reads the words **product** and **North** and turns them into arguments.
Python unpacks them with `func(**args)` – the same as calling the function by hand.

How does the LLM choose?

It compares the meaning of the question with each tool's description:

Question	Tool(s) requested
What were our total sales and profit?	`get_sales_summary`
Which product made the most profit?	`get_sales_by` {"group_by": "Product"}
What was our total profit in AED?	`get_sales_summary` + `get_exchange_rate` – two at once
What is the capital of France?	none – answers directly

The description IS the selection logic. A vague description gives wrong choices.

multi_tools.py

```
from tool_schemas import TOOLS

available_tools = {
    "get_sales_summary": get_sales_summary,
    "get_sales_by": get_sales_by,
    "get_exchange_rate": get_exchange_rate,
}
question = "What was our total profit in AED?"
messages = [{"role": "user", "content": question}]
response = client.chat.completions.create(model=MODEL, messages=messages, tools=TOOLS)
msg = response.choices[0].message

if msg.tool_calls:
    messages.append(msg)
    for call in msg.tool_calls:                                  # 0, 1 or more requests
        args = json.loads(call.function.arguments or "{}")   # the LLM fills the arguments
        result = available_tools[call.function.name](**args)
        messages.append({"role": "tool", "tool_call_id": call.id, "content": json.dumps(result)})
    final = client.chat.completions.create(model=MODEL, messages=messages)
```

Run it

```
> python multi_tools.py
```

```
LLM requested: get_sales_summary {}
LLM requested: get_exchange_rate {'currency': 'AED'}

Final answer:
 Total profit was $3,240, which is about AED 11,890.80 at 3.67 AED per USD.
```

Two tools in one reply. The LLM combined both results – neither tool could answer alone.

What have we learned?

Stage 5 adds choice:

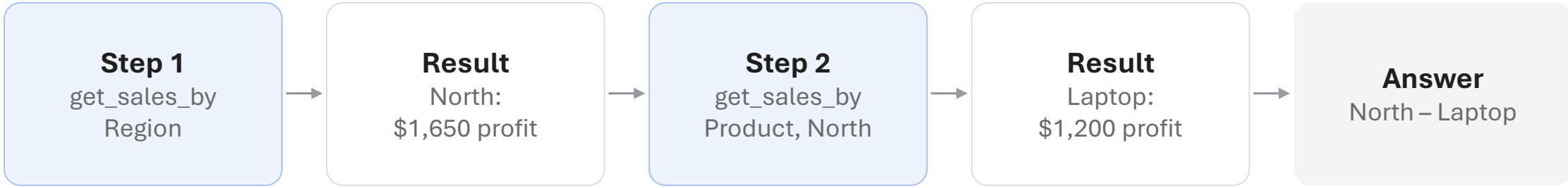
Element	Responsibility
Tool menu (`TOOLS`)	What the LLM can choose from – name, description, parameters
Description	How the LLM decides which tool fits
Parameters	The LLM fills them from the question; Python unpacks with `**args`
`for` loop	Handles 0, 1 or many tool requests in one reply

6. The agent loop

Repeat until the LLM has everything it needs.

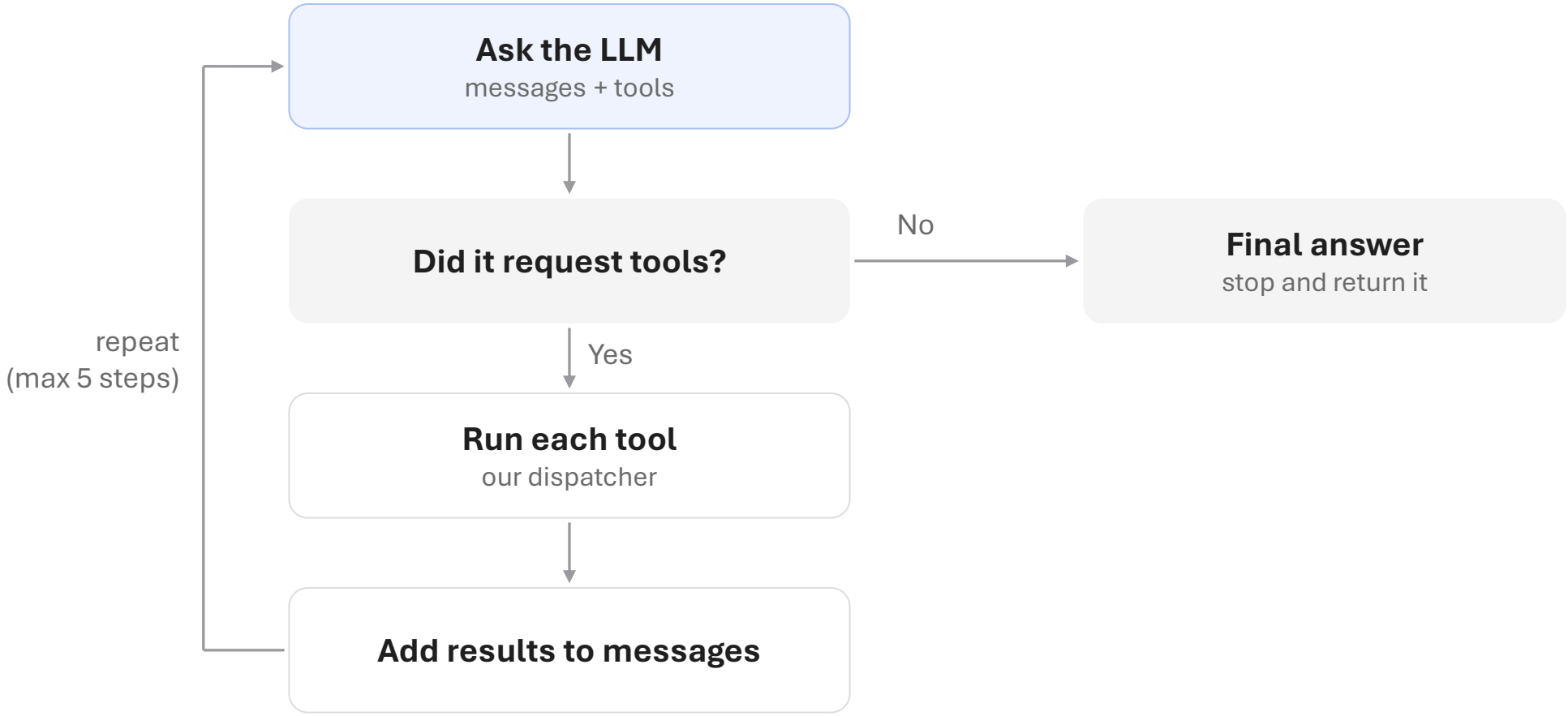
One round is not enough

"Which region made the most profit, and which product earned the most profit **in that region**?"



Step 2 needs **North** – and the LLM only learns that from the result of step 1.
Stage 5 code asks the LLM once, so it cannot do this. We need to ask again after every result.

The loop


Ask the LLM
messages + tools
Did it request tools?
No
Final answer
stop and return it
Yes
Run each tool
our dispatcher
Add results to messages
repeat
(max 5 steps)

agent.py

```
MAX_STEPS = 5   # safety limit: never loop forever


def run_agent(messages):
    for step in range(1, MAX_STEPS + 1):

        # 1. Ask the LLM, always with the tools available.
        response = client.chat.completions.create(model=MODEL, messages=messages, tools=TOOLS)
        msg = response.choices[0].message
        messages.append(msg)

        # 2. No tool requested -> this is the final answer. Stop.
        if not msg.tool_calls:
            return msg.content

        # 3. Run every requested tool and add its result to the conversation.
        for call in msg.tool_calls:
            args = json.loads(call.function.arguments or "{}")
            print(f"Step {step}: {call.function.name} {args}")
            result = available_tools[call.function.name](**args)
            messages.append({"role": "tool", "tool_call_id": call.id, "content": json.dumps(result)})

        # 4. Loop back: the LLM now sees the results and decides what is next.

    return "Stopped: too many steps without a final answer."
```

What changed from stage 5?

Stage 5 – one round	Stage 6 – a loop
Ask once	Ask (with tools)
Run the requested tools	Run the requested tools
Ask again WITHOUT tools	Ask again WITH tools
Must answer now	Repeat until no tool calls

Same building blocks – the only real change is who decides when we are done.

Run it

```
> python agent.py
```

```
Step 1: get_sales_by {'group_by': 'Region'}
Step 2: get_sales_by {'group_by': 'Product', 'region': 'North'}

Answer: The North region made the most profit ($1,650). In the North, Laptops
earned the most profit ($1,200).
```

Step 2 uses 'North' – a value the LLM learned from step 1. That is the agent loop at work.

What have we learned?

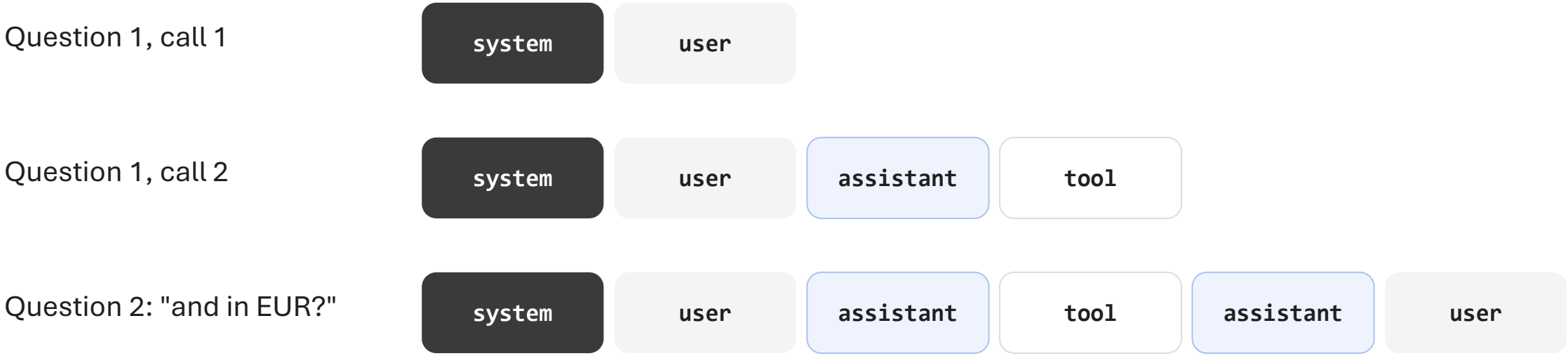
Stage 6 turns our program into an agent:

Element	Responsibility
The LLM	Decides what happens next: another tool or the final answer
The loop	Gives the LLM the results and asks again
`if not msg.tool_calls`	The normal way out: the answer is ready
`MAX_STEPS`	The safety way out: never loop forever

7. State and instructions

Instructions and memory across questions.

The LLM has no memory



Every API call is independent. The LLM only knows what is in the list we send.
To "remember" question 1, we simply keep the list and send it again.

Roles in the messages list

Role	Who writes it	Purpose
`system`	Us, once, at the top	Instructions: who the agent is and its rules
`user`	The person	The question
`assistant`	The LLM	Its answer – or a tool request
`tool`	Our dispatcher	The result of a tool the LLM requested

agent_chat.py

```
from agent import run_agent

# Instructions: who the agent is and the rules it must follow.
SYSTEM_PROMPT = """You are a sales analyst for our company.
- Use the tools for every number. Never guess figures.
- Show amounts in USD unless the user asks for another currency.
- Keep answers short: 2 to 4 sentences."""

# State: this list IS the agent's memory. We create it once and keep it.
messages = [{"role": "system", "content": SYSTEM_PROMPT}]

while True:
    question = input("\nYou: ").strip()
    if question.lower() in ("exit", "quit"):
        break
    messages.append({"role": "user", "content": question})
    answer = run_agent(messages)          # same loop as agent.py
    print("Agent:", answer)
```

Run it

```
> python agent_chat.py
```

```
You: What was our total profit?
Step 1: get_sales_summary {}
Agent: Total profit was $3,240 on $10,230 of revenue (31.67% margin).

You: and in EUR?
Step 1: get_exchange_rate {'currency': 'EUR'}
Agent: That is about €2,980.80 at 0.92 EUR per USD.
```

"and in EUR?" only makes sense because the first question and its answer are still in the list.

Proof: remove the memory

List created once (memory)

```
messages = [system]
while True:
    ...
```

"and in EUR?"
→ €2,980.80

List created inside the loop

```
while True:
    messages = [system]
    ...
```

"and in EUR?"
→ "EUR of what?"

Move one line and the agent forgets everything. Memory is our list – nothing more.

What have we learned?

Stage 7 adds instructions and memory:

Element	Responsibility
System message	Gives the agent its role and rules – no code change needed
`messages` list	The agent's memory: everything the LLM will see
Created once	Follow-up questions work because the history is kept
`run_agent`	Reused unchanged – stages stack on top of each other

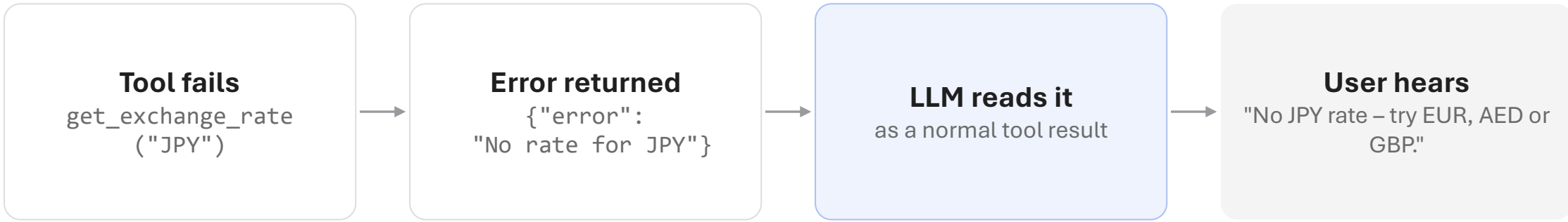
8. Make it reliable

Never crash, never loop forever, ask when unclear.

What can go wrong?

Problem	Real example	Our fix
Junk arguments	`{'': {}}` – crashed our script	Keep only the arguments the function accepts
Invalid JSON	`{not json`	Catch it, return an error
Unknown tool	`delete_database`	Allowlist check before running
Tool fails	Missing argument, missing file	`try/except` – send the error to the LLM
Endless loop	Keeps calling tools	`MAX_STEPS`, then stop honestly
Vague question	"Sales for last period?"	System rule: ask one clarifying question

Errors are data, not crashes



A crash ends the conversation. An error message lets the LLM explain, retry or ask.
Rule: the agent may fail a task – it must never crash.

agent_reliable.py – execute_tool()

```
def execute_tool(call):
    """Safe dispatcher: the agent must never crash because of the LLM."""
    name = call.function.name

    # 1. Only run tools we approved.
    if name not in available_tools:
        return {"error": f"Unknown tool '{name}'."}
    func = available_tools[name]

    # 2. Arguments must be valid JSON.
    try:
        args = json.loads(call.function.arguments or "{}")
    except json.JSONDecodeError:
        return {"error": "The tool arguments were not valid JSON."}

    # 3. Keep only the arguments the function really accepts.
    allowed = inspect.signature(func).parameters
    args = {k: v for k, v in args.items() if k in allowed}

    # 4. A failing tool returns an error to the LLM instead of crashing.
    try:
        return func(**args)
    except Exception as e:
        return {"error": f"{type(e).__name__}: {e}"}
```

agent_reliable.py – stop or ask

```
SYSTEM_PROMPT = """You are a sales analyst for our company.
- Use the tools for every number. Never guess figures.
- The data covers January to March 2026 only.
- If a question is unclear (which period, region or metric?), ask ONE short
  clarifying question instead of guessing.
- If a tool returns an error, explain it simply and suggest what to try.
- Keep answers short: 2 to 4 sentences."""

def run_agent(messages):
    for step in range(1, MAX_STEPS + 1):
        ...
        for call in msg.tool_calls:
            result = execute_tool(call)          # the safe dispatcher
            ...

    # Execution limit reached: stop and say so honestly.
    return "I could not finish within the step limit. Please rephrase or narrow the question."
```

Test plan

Try this	Expected behaviour
What was our total profit in AED?	Two tools in one step, answer in AED
Which region made the most profit, and its best product?	Two steps – step 2 uses step 1's result
What were sales last period?	Asks which period – no tool, no guess
What is our profit in JPY?	Tool error, explained, alternatives offered
Rename sales.csv, ask any question	File error explained – no crash
What is the capital of France?	Answers directly or redirects – no tool

Run it

```
> python agent_reliable.py
```

```
You: What were sales last period?
Agent: Which period do you mean? The data covers January to March 2026.

You: What is our profit in JPY?
  [step 1] get_sales_summary -> {'total_revenue': 10230.0, ... 'total_profit': 3240.0, ...}
  [step 1] get_exchange_rate -> {'error': "No rate available for 'JPY'."}
Agent: Total profit was $3,240. I don't have a JPY rate – I can show it in EUR, AED or GBP.
```

The agent asked instead of guessing, and explained the error instead of crashing.

What have we learned?

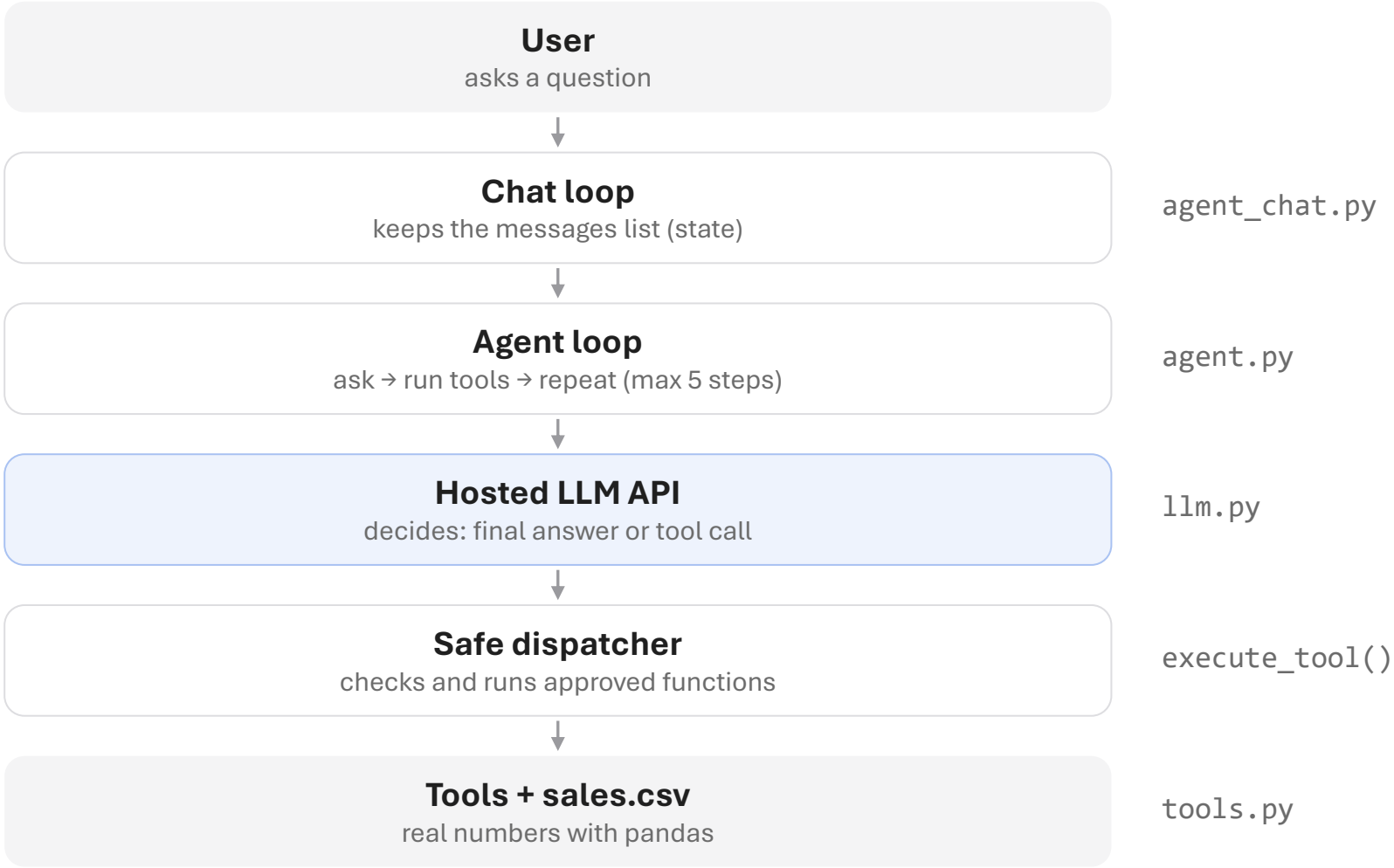
Stage 8 makes the agent safe to use:

Element	Responsibility
`execute_tool`	Allowlist, valid JSON, accepted arguments, errors returned
Errors as data	The LLM can explain or retry instead of the program crashing
`MAX_STEPS`	Guaranteed stop with an honest message
System rules	Ask when unclear; explain errors simply

9. Putting it all together

See the whole agent – and what comes next.

The complete agent



The six concepts – in our code

Concept	Where you saw it	File
LLM interaction	`messages`, roles, `client.chat.completions.create`	main.py, llm.py
Tools	name, description, parameters	tools.py, tool_schemas.py
Tool execution	`available_tools`, `func(**args)`	run_tool_call.py
The agent loop	`for step in range(MAX_STEPS)`, `if not msg.tool_calls`	agent.py
State	system message, list created once	agent_chat.py
Reliability	`execute_tool`, errors as data, step limit, clarify	agent_reliable.py

Project files

File	Stage	Role
`sales.csv`	2	Business data
`tools.py`	2, 5	Real calculations with pandas
`main.py`	1	First LLM call
`test_tool.py`	2	Test the tool without AI
`inspect_tool_call.py`	3	See the LLM's tool request
`llm.py`	1	Shared client and model
`run_tool_call.py`	4	Execute the tool, return the result
`tool_schemas.py`	5	The tool menu for the LLM
`multi_tools.py`	5	Choices and parameters
`agent.py`	6	The agent loop
`agent_chat.py`	7	Instructions and memory
`agent_reliable.py`	8	The complete, safe agent

Beyond the basics

Long-term memory
Remember facts between sessions

Knowledge (RAG)
Answer from documents, not only data

MCP
Share our tools with any AI app

Human approval
Ask before actions like sending email

Observability
Log steps, tokens and cost

Frameworks
Same six concepts, packaged

Each one is built on the agent you now understand.

Questions?

AI Agent from Scratch | Naji El Kotob